

SIMATS ENGINEERING
CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192524358
Name	Abdul Razak Shaik

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I ___Abdul Razak Shaik (192524358)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Shaik .Abdul Razak

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50

1.AI Drone Pathfinding in a Flood-Affected Region

i. Greedy Best-First Search (GBFS) Behaviour, Strengths, and Weaknesses

Behaviour:

- Greedy Best-First Search selects the path that appears closest to the destination using only the heuristic value **$h(n)$** (straight-line distance).
- It ignores the actual travel cost caused by weather, terrain, or blocked roads.
- The drone quickly moves toward the goal based on the estimated shortest direction.

Strengths:

- Very fast and requires less computation.
- Suitable for urgent situations where a quick route is needed.
- Efficient in large search spaces.

Weaknesses:

- Does not guarantee the shortest or safest route.
- May choose blocked or risky paths because it ignores movement cost.
- May need to backtrack frequently when obstacles appear.
- Performance decreases in highly dynamic environments.

ii. A* Search in the Same Scenario

Behaviour:

- A* evaluates each path using:

$$f(n) = g(n) + h(n)$$

Where:

- **$g(n)$** = Actual travel cost (terrain, weather, distance)
- **$h(n)$** = Estimated distance to the destination

How it Balances Cost and Heuristic:

- Considers both the actual path cost and estimated remaining distance.
- Avoids expensive or dangerous routes while still moving toward the goal.
- Updates the path whenever real-time information about blocked roads is received.

Advantages:

- Finds the optimal path if the heuristic is admissible.
- Produces safer and more reliable routes.
- Better suited for environments with varying travel costs.

iii. Impact of Heuristic Accuracy

Heuristic Accuracy	Greedy Best-First Search	A* Search
Highly Accurate	Fast and usually reaches the goal quickly	Finds the optimal path efficiently
Moderately Accurate	May choose suboptimal paths	Still performs well with small errors
Poor/Inaccurate	Frequently selects wrong directions	Performance decreases but still considers actual costs

Observation:

- GBFS depends almost entirely on heuristic accuracy.
- A* is less sensitive because it also considers the accumulated travel cost.

iv. Effect of Dynamic Changes (Blocked Paths)

Greedy Best-First Search:

- May continue toward blocked paths.
- Requires repeated replanning.
- Can waste time due to backtracking.

A Search: *

- Recalculates path using updated costs and obstacles.
- Avoids blocked routes more effectively.
- Maintains better route quality despite environmental changes.

Overall Impact:

- Dynamic obstacles increase computation for both algorithms.
- A* adapts more efficiently because it evaluates total path cost.

v. Recommended Algorithm

Recommended Algorithm: A*

Justification:

- Produces the safest and most reliable route.
- Considers both actual travel cost and estimated distance.
- Adapts better to changing weather and blocked roads.
- Guarantees the optimal path when the heuristic is admissible.
- Although slightly slower than Greedy Best-First Search, it provides better safety and lower overall delivery delay in emergency situations.

2.Smart City Traffic Signal Optimization:

Formulation of the Optimization Problem:

The AI system aims to optimize traffic signal timings at hundreds of intersections to minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

Objective Function:

Minimize:

$$f(x) = \text{Waiting Time} + \text{Fuel Consumption} + \text{Traffic Congestion}$$

where x represents the signal timing configuration for all intersections.

Why Traditional Search is Inefficient:

- The number of possible signal timing combinations is extremely large.
- Traffic conditions change continuously.
- Exhaustive search requires excessive computation.
- Real-time traffic management demands quick decision-making.

i. Hill Climbing in This Scenario:

Working:

- Start with an initial traffic signal timing.
- Generate neighbouring solutions by slightly changing signal timings.
- Select the neighbouring solution with the best improvement.
- Repeat until no better solution is found.

Limitations:

- Can get trapped in local optimum solutions.
- Cannot escape flat regions (plateaus).
- May fail to find the global optimum.
- Performance depends heavily on the starting solution.

ii. Local Maxima, Plateaus, and Ridges (Real-World Interpretation):

Local Maxima:

- The system finds a traffic signal configuration that is better than nearby solutions but not the best overall.
- Example: One busy intersection improves while congestion shifts to nearby roads.

Plateaus:

- Many neighbouring signal timings produce nearly identical traffic performance.
- Example: Small timing changes do not noticeably reduce waiting time.

Ridges:

- The best improvement requires changing several signal timings simultaneously.
- Example: Adjusting one junction alone gives no improvement, but coordinating multiple intersections reduces congestion significantly.

iii. Simulated Annealing and Genetic Algorithms:

Simulated Annealing:

Working:

- Sometimes accepts worse solutions with a certain probability.
- This helps escape local optima.
- As the temperature decreases, the algorithm becomes more selective.

Advantages:

- Escapes local maxima.
- Produces better global solutions.
- Suitable for dynamic optimization problems.

Genetic Algorithm:

Working:

- Represents traffic signal plans as chromosomes.
- Selects the best-performing solutions.
- Applies crossover and mutation to generate improved solutions.
- Repeats over multiple generations.

Advantages:

- Explores many solutions simultaneously.
- Avoids premature convergence.

- Highly scalable for large optimization problems.
- Easily adapts to changing traffic conditions.

iv. Recommended Approach

Recommended Algorithm: Genetic Algorithm (GA):

Justification:

- Handles very large search spaces efficiently.
- Optimizes signal timings across hundreds of intersections simultaneously.
- Adapts well to changing traffic patterns.
- Less likely to become trapped in local optima.
- Provides high-quality solutions within reasonable computation time.
- Well suited for real-time smart city traffic optimization.

3. Autonomous Mars Rover – Online Search Agent:

i. Why This Problem Requires an Online Search Agent:

The Mars rover has no complete map of the terrain and discovers obstacles while moving. Therefore, it cannot plan the entire route in advance.

Why Online Search?

- Explores and searches simultaneously.
- Makes decisions using current sensor information.
- Continuously updates its path as new terrain is discovered.
- Suitable for unknown and real-time environments.

Why Offline Search is Not Suitable?

- Requires complete knowledge of the environment before planning.
- Cannot easily adapt to newly discovered hazards.
- Not practical for Mars exploration where the map is incomplete.

ii. Challenges of Partial Observability and Dynamic Environments:

Partial Observability:

- The rover can sense only nearby terrain.
- Hidden obstacles such as rocks or craters may not be detected until closer.
- Decisions must be made with incomplete information.

Dynamic Environment:

- New hazards may appear during exploration.
- Terrain conditions may change due to dust storms or loose soil.
- The rover must frequently re-evaluate its path.

Challenges:

- Increased uncertainty.
- Frequent replanning.
- Risk of choosing unsafe routes.

iii. How the Rover Builds and Updates Its Internal Model:

The rover maintains an internal map of the environment.

Process

1. Sense nearby terrain using onboard sensors.
2. Detect obstacles and safe paths.
3. Update the internal map with newly discovered information.
4. Recalculate the best route if hazards are found.
5. Repeat until the destination is reached.

This continuous learning allows the rover to improve navigation over time.

iv. Comparison of Online Search with Uninformed and Informed Search:

Feature	Online Search	Uninformed Search	Informed Search
Environment Knowledge	Partial	Complete	Complete
Uses Heuristic	Optional	No	Yes
Decision Making	During exploration	Before exploration	Before exploration
Adapts to New Information	Yes	No	Limited
Suitable for Mars Rover	Yes	No	Partially

v. Recommended Approach:

Recommended Method: Online Search Agent

Justification

- Operates effectively without a complete map.
- Continuously updates its knowledge using sensor data.
- Adapts quickly to newly discovered hazards.
- Ensures safer navigation in uncertain environments.
- Supports real-time decision making and exploration

4. University Exam Timetable as a Constraint Satisfaction Problem (CSP):

Problem Formulation as a CSP:

The exam scheduling problem can be represented as a Constraint Satisfaction Problem (CSP) where the objective is to assign exam time slots while satisfying all constraints.

i. Variables, Domains, and Constraints:

Variables

Each variable represents a course exam.

Example:

- X_1 = Mathematics
- X_2 = Physics
- X_3 = Chemistry
- X_4 = Computer Science

Domains

The domain of each variable is the set of available exam time slots and halls.

Example:

- $D(X) = \{\text{Day1-Morning, Day1-Afternoon, Day2-Morning, Day2-Afternoon}\}$

Constraints

1. **No Overlapping Exams**
 - Students cannot have two exams at the same time.
2. **Limited Exam Halls**
 - The number of exams scheduled simultaneously cannot exceed available halls.
3. **Subject Ordering**
 - Some subjects must be scheduled before others.
4. **Faculty Availability**
 - Exams must be scheduled only when faculty members are available.
5. **Special Accommodations**
 - Students requiring extra facilities must be assigned suitable halls.

These constraints ensure a valid and conflict-free timetable.

ii. Backtracking Search:

Working

1. Select an unscheduled exam.
2. Assign a valid time slot.
3. Check whether all constraints are satisfied.

4. If a conflict occurs, undo the assignment (backtrack).
5. Try another time slot.
6. Continue until all exams are scheduled.

Advantages

- Guarantees a valid solution if one exists.
- Simple and systematic.

Limitation

- Can become slow for large universities due to many possible combinations.

iii. Constraint Propagation (Forward Checking):

Working

- After assigning a time slot, remove conflicting time slots from the domains of remaining exams.
- Detect conflicts early before exploring unnecessary branches.

Advantages

- Reduces the search space.
- Prevents invalid assignments.
- Speeds up timetable generation.
- Improves scalability compared to plain backtracking.

iv. Real-World Challenges and Best Strategy:

Challenges

- Thousands of students and courses.
- Frequent timetable updates.
- Multiple resource constraints.
- Last-minute faculty changes.
- Limited halls and special accommodations.

Recommended Strategy

Backtracking + Forward Checking (Constraint Propagation)

Justification

- Efficiently eliminates invalid choices.
- Reduces computation time.
- Handles complex constraints effectively.
- Suitable for large-scale university scheduling.

5. Strategic Game AI (Minimax & Alpha-Beta Pruning):

i. How Minimax Models the Problem:

Minimax models the game as a decision tree.

- MAX node: AI chooses the move with the highest score.
- MIN node: Human opponent chooses the move with the lowest score for the AI.

The algorithm assumes both players make optimal decisions.

Steps

1. Generate possible moves.
2. Expand the game tree.
3. Evaluate leaf nodes.
4. Propagate scores upward.
5. Select the best move.

ii. Computational Challenges of Large Game Trees

Challenges

- Huge branching factor.
- Very deep search trees.
- High memory usage.
- Slow decision-making.
- Difficult to meet real-time constraints.

Searching every possible move becomes computationally expensive.

iii. How Alpha-Beta Pruning Improves Efficiency

Working

Alpha-Beta pruning removes branches that cannot affect the final decision.

- Alpha (α): Best score found for the MAX player.
- Beta (β): Best score found for the MIN player.

If $\alpha \geq \beta$, the remaining branches are pruned because they cannot improve the outcome.

Advantages

- Reduces the number of nodes evaluated.
- Produces the same optimal result as Minimax.
- Allows deeper searches within the same time.
- Improves real-time performance significantly.

iv. Evaluation Function:

The evaluation function estimates the quality of a game state.

Example

Evaluation Score

$$\begin{aligned} &= (5 \times \text{Resources}) + (10 \times \text{Army Strength}) \\ &+ (8 \times \text{Territory Control}) + (6 \times \text{Defense}) \\ &- (7 \times \text{Enemy Strength}) \end{aligned}$$

Factors Considered

- Available resources.
- Army strength.
- Territory controlled.
- Defensive position.
- Enemy power.
- Remaining health or units.

These factors help the AI choose strong strategic moves.

v. Recommended Strategy:

Recommended Approach

Minimax with Alpha-Beta Pruning

Justification

- Produces optimal decisions.
- Eliminates unnecessary search branches.
- Handles large game trees efficiently.
- Meets real-time decision-making requirements.
- Widely used in modern game AI.